আন্তর্জাতিক ইসলামী বিশ্ববিদ্যালয় চট্টগ্রাম

الجامعة الإسلامية العالمية شيتاغونغ

International Islamic University Chittagong

Department of Computer Science an Engineering

Project Report

Portfolio Application Using Flutter and GetX

Course Code : CSE-3644

Course Title : Software Development II Lab

Name : Md. Jamil Siddique Chowdhury

ID : C233166

Section : 6EM

Semester : 6th

Submitted to : Shahriar Rahman Inan

Adjunct Lecturer, Dept of CSE, IIUC

Remarks

Introduction:

For this project, I built a mobile portfolio app in Flutter that showcases who I am as a developer — my background, my skills, and the projects I've worked on so far. The idea was simple: instead of a static resume or a plain webpage, why not have something interactive that recruiters, teachers, or potential clients can actually open on their phone and scroll through?

I built the app using Flutter, and for managing state and organizing the code I went with the GetX architecture. Beyond just getting the app working, I wanted it to look and feel polished, so a good part of the effort went into responsive layouts, smooth animations, and writing code that's actually reusable rather than copy-pasted across screens — all things we covered in the SD2 Lab course.

Project Features

The app is split into five main sections, each covering a different part of what a visitor would want to know.

1.Home Section

This is the first thing anyone sees when they open the app. I kept it simple on purpose — a header, my profile picture, my name, and my designation as an Aspiring Software Engineer, all wrapped in a layout that adjusts itself to whatever screen size it's opened on. A few subtle animations run here too, just enough to make the first impression feel alive rather than static.

- Portfolio header
- Professional profile image
- Full name
- Professional designation (Aspiring Software Engineer)
- Modern responsive layout
- Animated interface elements

2. About Me Section

Here's where I introduce myself properly — a short bio, my career objective, my educational background, the kind of work I'm interested in, and a brief self-introduction. This is the part that gives visitors an actual sense of who I am beyond a list of skills.

- Personal biography
- Career objective

- Educational background
- Professional interests
- Short self-introduction

3. Skills Section

Instead of just listing technologies in plain text, I built this section around animated cards — each one responds when you tap or hover over it. The technologies currently listed are:

- ASP.NET Core
- Flutter
- Razor
- C#
- JavaScript
- Microsoft SQL Server
- MySQL
- REST API
- Git

Each card has its own icon, and the little hover/tap animation gives the whole section a more interactive feel instead of it reading like a static resume line.

4. Projects Section

This is probably the section I put the most thought into, since it's what actually demonstrates my work. Every project gets its own card with an image, a title, a short description, and the tech stack used — plus a bit of interactivity so it doesn't feel like a flat list.

- Project image
- Project title
- Short description
- Technology stack
- Interactive design

Right now there are two projects featured:

School Management System

Built with ASP.NET Core, this one handles student records and general administrative tasks for a school — the kind of system that manages data instead of just displaying it.

Task Planner Application

A straightforward task management app aimed at helping users keep track of their daily to-dos and stay a bit more productive.

Both cards use the same UI patterns — elevation, shadows, and animation — to keep the section feeling consistent.

5. Contact Section

The last section is just about making it easy for someone to actually reach out — email, phone number, location, and links to my GitHub and LinkedIn, each behind a clickable, animated icon.

- Email address
- Phone number
- Location
- GitHub profile
- LinkedIn profile

Animation Implementation:

Animation was actually one of my main goals going into this project — I didn't want a plain, static UI, so I spent a fair amount of time getting size and color transitions to feel smooth across the different components.

Navigation Bar Animation

Tapping the menu icon or moving between navigation items triggers a small animated transition rather than an instant switch.

- Color transition
- Smooth highlighting
- Interactive touch feedback

Profile Image Animation

The profile picture on the home screen isn't just a static image — it's wrapped in animated widgets that give it a bit of life when the page loads.

- Size animation
- Circular border effect
- Shadow animation
- Smooth scaling

Skill Card Animation

Each skill card reacts when tapped — it expands slightly, the background and border shift color, and the whole card scales up a bit.

- Card expansion
- Background color transition
- Border color transition
- Scale effect

Whichever card is currently active gets a glowing border so it's obvious at a glance which one you're looking at.

Project Card Animation

The project cards use elevation and shadow changes along with animated buttons to give a sense of depth when interacted with.

- Card elevation
- Shadow transition
- Interactive button animation
- Smooth hover/tap response

Contact Button Animation

Even the social/contact buttons got the same treatment — a small scale animation and color shift on tap so the whole app feels consistent.

- Circular scale animation
- Color transition
- Interactive feedback
- Smooth icon response

GetX Architecture Implementation:

Rather than throwing everything into one big file, I structured the whole project around GetX, which naturally splits things into a few clear layers. It made the codebase a lot easier to navigate once the project started growing.

Controllers

These handle the logic side of things — UI state, animation states, navigation, and reacting to whatever the user does.

- UI state
- Animation states
- Navigation
- User interactions

Views

The actual screens of the app live here:

- Home Screen
- About Section
- Skills Section
- Projects Section
- Contact Section

Widgets

Instead of rewriting similar UI code over and over, I pulled the repeated pieces into their own reusable widgets:

- Skill cards
- Project cards
- Contact buttons
- Section titles
- Navigation components

This alone saved a lot of time and made the code far less repetitive.

Models

Data like skills, projects, and personal/contact details are structured through models rather than scattered around as loose variables.

- Skills
- Projects
- Personal information
- Contact information

Reactive State Management

This is where GetX really shines — using GetxController, .obs observable variables, and Obx widgets, the UI updates on its own whenever the underlying data changes, without needing to rebuild the entire screen.

- GetxController
- Observable (.obs) variables
- Obx widgets

Dependency Injection

Controllers are initialized through GetX's dependency injection, which keeps memory usage down and makes managing controllers across the app much simpler.

- Reduces memory usage
- Simplifies controller management
- Improves scalability

Routing

Navigation between screens is handled through GetX routing, which kept things organized without needing a separate navigation setup.

Reflection of SD2 Lab Learning:

Looking back, this project ended up being a pretty good summary of everything I picked up in SD2 Lab, all put into practice at once.The course is where I really learned how to build cross-platform apps with Flutter and Dart in the first place. Widgets like Scaffold, Container, Row, Column, Stack, ListView, Text, Image, Icon, and ElevatedButton went from things I'd just read about to things I was actually using constantly to piece together each screen of the portfolio.Responsive design was another area I got a lot more comfortable with. Getting a layout to hold up across different screen sizes without looking broken or squeezed took some trial and error, but the app follows those principles fairly consistently now.Animation is probably where I spent the most time experimenting. Using AnimatedContainer and similar widgets, I worked through size changes, color transitions, and scaling effects until they felt smooth rather than jumpy — and honestly, that back-and-forth tweaking taught me more than just reading the documentation ever could.

GetX state management was probably the single most useful thing I took from this course. Getting GetxController, observable variables, Obx widgets, dependency injection, and routing to work together meant I could keep the app's logic separate from its UI — which made the whole project noticeably easier to manage as it grew.On top of that, organizing everything into widgets, controllers, and models pushed me to think more carefully about clean code in general, not just for this project but as a habit going forward. And using Git and GitHub throughout kept the whole process manageable and easy to track.All together, this project brought Flutter development, responsive design, animation, reusable components, and

GetX architecture into one place — which is really what SD2 Lab was building toward the whole time.

Conclusion:

In the end, the Interactive Developer Portfolio Application does what I set out to build — a working, well-animated Flutter app built on the GetX architecture that holds up across different screen sizes.It presents my profile, skills, projects, and contact details in a way that's clean to look at and easy to navigate, and building it pushed my understanding of Flutter widgets, state management, and animation a lot further than just following along in class would have.More than anything, it's a project I can keep building on — adding new projects, certifications, and skills as I pick them up, rather than something that's finished and set aside.